Universität Rostock




# ANFORDERUNGEN

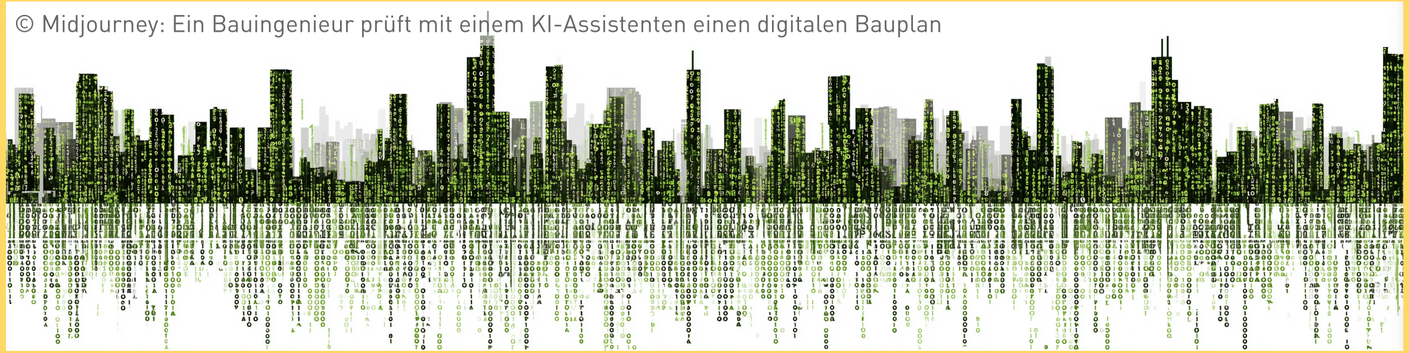
Vom Problem zum Prompt

Joern Ploennigs · AI4SC

*Midjourney: A civil engineer reviewing a digital blueprint with an AI assistant*

# Hörsaalfrage

Wer legt in einem Projekt fest, was gebaut wird?


© Midjourney: Ein Bauingenieur prüft mit einem KI-Assistenten einen digitalen Bauplan

# WARUM ANFORDERUNGEN?

# KI erfüllt den Prompt, nicht den Bedarf

## Was KI tut

Sie erzeugt Code, der den Prompt erfüllt.

## Was KI nicht tut

Sie prüft nicht, ob der Prompt das Problem des Stakeholders trifft.

## Konsequenz

Sie sind die Brücke zwischen Stakeholder-Bedarf und KI-generiertem Code.

Keine gute Anforderung, kein guter Prompt.

# Anforderung: Quelle von Prompt und Test

Der agentische Zyklus:

01 **Prompt** → KI generiert Code

02 **Generate** → Code entsteht

03 **Understand** → Code lesen

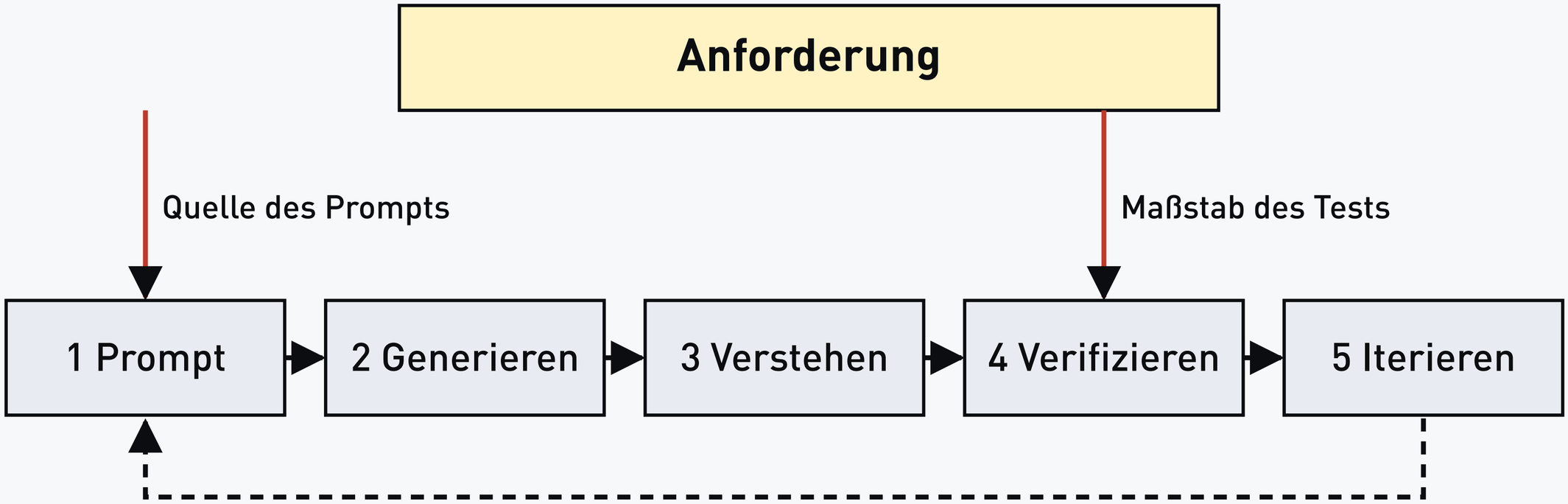
04 **Verify** → Testen gegen Anforderung

05 **Iterate** → Prompt verfeinern

**Anforderung** ist Grundlage für Schritt 1 und Schritt 4.

Ohne klare Anforderung:

- kein präziser Prompt
- kein sinnvoller Test
- kein Maßstab für „fertig"



Die Anforderung speist Schritt 1 (Prompt) und Schritt 4 (Verifizieren). Ohne sie gibt es kein Maß für „fertig“.

# ANFORDERUNGSARTEN

# Drei Kategorien

## Funktional

Was das System tun muss

- Messwerte einlesen
- Grenzwert prüfen
- Alarm auslösen
- Bericht erzeugen

## Nichtfunktional

Wie gut es das tun muss

- Antwortzeit unter 2 s
- 1 Mio. Datensätze verarbeiten
- Ausgabe IFC-konform

## Randbedingungen

Womit und worunter

- Raspberry Pi 4
- Python 3.11
- DSGVO

# Beispiel: Sensorüberwachung Brückenbauwerk

## Szenario

Überwachungssystem für Feuchte- und Temperatursensoren in einem Brückenbauwerk.

## Funktional

- Messwerte alle 60 s einlesen
- Grenzwertüberschreitung erkennen
- Alarm per E-Mail versenden
- Messdaten lokal speichern

## Nichtfunktional

- Antwortzeit unter 2 s
- Weiterbetrieb bei Netzausfall

## Randbedingungen

- Hardware: Raspberry Pi 4
- Betriebssystem: Raspberry Pi OS

# Häufige Fehler bei Anforderungen

## Schlecht

- Vage: „Das System soll schnell sein“
- Nicht testbar: „Benutzerfreundliche Oberfläche“
- Lösungsorientiert: „Nutze SQLite als Datenbank“

## Besser

+ Messbar: „Antwortzeit unter 2 s bei 95 % der Anfragen“
+ Testbar: „Die Alarmfunktion wird ohne Anleitung in unter 30 s gefunden“
+ Problemorientiert: „Messdaten bleiben nach Stromausfall abrufbar“

# STAKEHOLDER-PERSPEKTIVEN

# Drei Perspektiven, ein System

## Tragwerksplaner

Braucht:

- Rohmesswerte (kPa, °C)
- Langzeittrends
- Export in IFC / CSV

## Facility Manager

Braucht:

- Echtzeit-Alarmierung
- Dashboard auf dem Tablet
- Eskalationsregeln

## Behördenprüfer

Braucht:

- Audit-Trail
- Manipulationssicherheit
- Exportierbare Berichte

# Stakeholder-Bedarf in Anforderungen überführen

## Vorgehen

01 Stakeholder identifizieren

02 Nutzungsszenarien je Rolle erfassen

03 Funktionale und nichtfunktionale Anforderungen ableiten

04 Konflikte aufdecken und priorisieren

## Typische Konflikte

- Echtzeit (Manager) gegen Langzeitarchiv (Planer)
- Offene API gegen Sicherheitsanforderung (Prüfer)
- Detailtiefe gegen Übersichtlichkeit

# QUALITÄTSKRITERIEN

# Messbare Abnahmekriterien

## „Das System soll zuverlässig sein.“

Messbar gemacht:

- Messwerte bleiben nach Stromausfall abrufbar
- Automatischer Neustart nach unter 60 s

## „Das System soll skalierbar sein.“

Messbar gemacht:

- 1 Mio. Datensätze in unter 10 s verarbeiten
- Ausgabe validiert gegen das IFC-Schema

> **Merksatz**
>
> Sofern möglich braucht jede nichtfunktionale Anforderung ein quantitatives Akzeptanzkriterium. Falls das nicht sinnvoll ist, braucht sie wenigstens ein klares Prüfkriterium.

# USER STORIES

# User-Story-Format

## Schablone

> Als **[Rolle]**
> möchte ich **[Funktion]**
> damit ich **[Nutzen]**.

Akzeptanzkriterien beantworten: Wann ist die Story erledigt?

## Beispiel

> Als **Facility Manager**
> möchte ich ein **Sensor-Alarm-Dashboard**
> damit ich **vor einem SLA-Bruch reagieren kann**.

- Dashboard lädt in unter 2 s
- Aktive Alarme werden rot hervorgehoben
- Quittierung wird mit UTC-Zeitstempel gespeichert

# ANFORDERUNG UND PROMPT

# Von der User Story zum Prompt: Kontext

## User Story

> Als Facility Manager
> möchte ich ein Sensor-Alarm-Dashboard
> damit ich vor einem SLA-Bruch reagieren kann.

- Lädt in unter 2 s
- Aktive Alarme rot markiert
- Quittierung mit UTC-Zeitstempel

## Prompt: Kontext-Abschnitt

```
Du bist ein Python-Entwickler.
Aufgabe: Sensor-Alarm-Dashboard.
Rolle: Facility Manager
Ziel: SLA-Überwachung in Echtzeit
```

Rolle und Nutzen der User Story bilden den Kontext des Prompts.

# Von der User Story zum Prompt: Anforderungen

## Akzeptanzkriterium zu Prompt-Bedingung

- Lädt in unter 2 s wird zu `Ladezeit < 2 s`
- Alarme rot wird zu `Alarme mit Status "active" rot`
- Zeitstempel wird zu `Quittierung + UTC-Zeitstempel`

## Vollständiger Prompt

```
Erstelle ein Streamlit-Dashboard
für Sensor-Alarme (Python).

Anforderungen:
- Ladezeit < 2 s
- Aktive Alarme rot hervorheben
- Quittierung + UTC-Zeitstempel

Schreibe pytest-Tests, die diese
Anforderungen prüfen.
```

**Merksatz**

Eine präzise Anforderung ist der beste Prompt.

# VON ANFORDERUNGEN ZU TESTS

# Von der Anforderung zum Test

## „Dashboard lädt in unter 2 s“

```python
def test_dashboard_load_time():
    start = time.time()
    dashboard = load_dashboard(
        sensor_data
    )
    elapsed = time.time() - start
    assert elapsed < 2.0, \
        f"Zu langsam: {elapsed:.2f}s"
```

## „Quittierung mit UTC-Zeitstempel“

```python
def test_alarm_acknowledgment():
    alarm = Alarm(
        sensor_id="S01",
        level="critical"
    )
    alarm.acknowledge(user="jdoe")
    assert alarm.ack_timestamp \
        is not None
    assert alarm.ack_user == "jdoe"
```

**Wenn die KI einen Test generiert**

01 Prüfe, ob der Test das Akzeptanzkriterium tested und nicht nur die Implementierung.

02 Führe den Test gegen die Anforderung aus, nicht nur gegen den Code.

03 Schreibe den Test vor dem Prompt, dann siehst du sofort, was fehlt.

04 Nutze bei nicht direkt testbaren Anforderungen ergänzend Inspektion, Analyse oder Demonstration.

# Test-Driven Prompting

## Reihenfolge

01 Anforderung formulieren

02 Akzeptanzkriterien ableiten

03 Test schreiben (noch kein Code)

04 Prompt aus Anforderung und Test

05 KI generiert Code

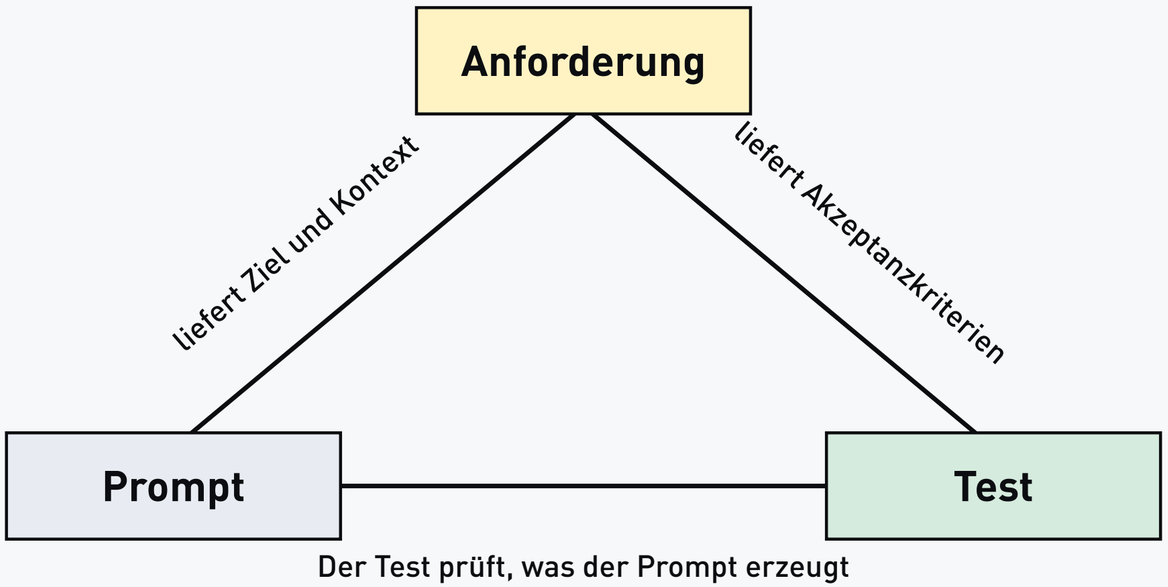
06 Test grün, Anforderung erfüllt

## Vorteil

- Der Prompt enthält den Test als Spezifikation
- Die KI hat ein klares Zielkriterium
- Verifikation ist objektiv und automatisierbar
- Anforderung, Prompt und Test bleiben synchron

# DAS TRACEABILITY-DREIECK

# Traceability-Dreieck




## Regel

Ändert sich die Anforderung, müssen Prompt und Test angepasst werden.

Ändert sich der Test, müssen Anforderung und Prompt geprüft werden.

Kein Test und kein Prompt ohne Anforderung.

**Merksatz**

Das Dreieck ist vollständig, wenn jede Anforderung einen Prompt und einen Test hat und jede Änderung alle drei aktualisiert.

# QUESTIONS